Lab 08: SonarQube — rentgen kodu

Automatyczna inspekcja jakości

Jarosław Hryszko
jaroslaw.hryszko@uj.edu.pl

Instytut Informatyki i Matematyki Komputerowej UJ

Otwarte repozytoria kodu i pomiary oprogramowania

Ręczne skrypty vs platforma

Do tej pory (laby 4–7)

- LOC, złożoność, metryki OO
- Code churn, truck factor
- Własne skrypty w Pythonie
- Wyniki: CSV, wykresy

SonarQube (dziś!)

- Wszystko powyższe + więcej
- Bugs, vulnerabilities, code smells
- Dashboard w przeglądarce
- Integracja z CI/CD

SonarQube to **platforma do ciągłej inspekcji jakości kodu** — open-source, 30+ języków.

Efekty kształcenia: W1, U2, K1

Architektura SonarQube

Dwa komponenty

SonarQube Server Serwer webowy z dashboardem, bazą danych, silnikiem reguł

Sonar Scanner Klient CLI — analizuje kod i wysyła wyniki do serwera

- Serwer działa cały czas (u nas: Docker)
- Scanner uruchamiany **per analiza** (u nas: też Docker)
- Komunikacja: REST API (scanner → serwer)
- Wyniki: przeglądarka na `http://localhost:9000`

Co analizuje SonarQube?

Reliability

- Bugs — błędy w kodzie
- Rating A–E

Security

- Vulnerabilities — luki
- Security Hotspots — do przeglądu
- Rating A–E

Maintainability

- Code Smells — problemy utrzymania
- Technical Debt (czas naprawy)
- Rating A–E

Inne

- Duplikacje kodu (%)
- Pokrycie testami (%)
- Złożoność, LOC

Quality Gate

Definicja

Quality Gate = zestaw warunków, które kod musi spełnić, żeby "przejść".

- Domyślny: **Sonar way** — skupia się na **nowym kodzie**
- Warunki np.: 0 nowych bugów, 0 nowych vulnerabilities, pokrycie $> 80\%$
- Wynik: **Passed** lub **Failed**
- W CI/CD: failed Quality Gate = zablokowany merge

Można tworzyć własne Quality Gates z własnymi progami.

Plan zajęć

1. **Zadanie 1 — Stawiamy SonarQube** (45 min)
   - Docker Compose: serwer + scanner
   - Konfiguracja projektu, token, pierwsza analiza
2. **Zadanie 2 — Pierwsza analiza** (60 min)
   - Czytanie dashboardu: bugs, vulnerabilities, code smells
   - Metryki: LOC, złożoność, duplikacje
   - Porównanie z wynikami z poprzednich labów
3. **Zadanie 3 — Własny Quality Gate** (30 min, opcjonalne)
   - Ostrzejsze progi niż Sonar way
   - Sprawdzenie czy projekt przechodzi

Co oddajecie?

W branchu `lab08_nazwisko1_nazwisko2`:

1. `answers.md` — odpowiedzi na pytania + metryki
2. `docker-compose.yml` — konfiguracja SonarQube
3. *(opcjonalnie)* opis własnego Quality Gate

Do roboty!

Instrukcja: `README.md` w repozytorium

"SonarQube to jak wizyta u dentysty — wiesz, że musisz iść, nie chcesz znać wyników, ale potem jest lepiej." — anonimowy developer